

DML Commands — INSERT, UPDATE, DELETE, MERGE

Chapter 11 | *PostgreSQL Complete Guide* Module: 02_SQL_Basics | Difficulty: Beginner | Estimated Reading Time: 35 minutes

Table of Contents

- [Learning Objectives](#)
 - [Theory: What is DML?](#)
 - [INSERT — Adding Data](#)
 - [UPDATE — Modifying Data](#)
 - [DELETE — Removing Rows](#)
 - [MERGE — Upsert Operations](#)
 - [RETURNING Clause](#)
 - [ASCII Visual Diagrams](#)
 - [Practical Examples](#)
 - [PostgreSQL Commands](#)
 - [SQL Examples](#)
 - [Common Mistakes](#)
 - [Best Practices](#)
 - [Performance Considerations](#)
 - [Interview Questions](#)
 - [Interview Answers](#)
 - [Hands-on Exercises](#)
 - [Solutions](#)
 - [Advanced Notes](#)
 - [Cross-References](#)
-

Learning Objectives

By the end of this chapter, you will be able to:

- Use INSERT to add single and multiple rows, including inserting from subqueries
 - Use ON CONFLICT for idempotent upserts (INSERT ... ON CONFLICT DO UPDATE)
 - Write targeted UPDATE statements with joins and subqueries
 - Use DELETE with WHERE, JOINS, and RETURNING to safely remove data
 - Implement MERGE (PostgreSQL 15+) for full upsert/delete in one statement
-

Theory: What is DML?

DML (Data Manipulation Language) is the SQL subset used to **create, read, update, and delete data** within database objects.

Command	Purpose	Notes
SELECT	Read/query data	Covered in next chapter
INSERT	Add new rows	Covered here
UPDATE	Modify existing rows	Covered here

DELETE	Remove rows	Covered here
MERGE	Conditional upsert	PostgreSQL 15+; covered here

DML vs. DDL

- **DDL** changes the **structure** (tables, columns, indexes)
- **DML** changes the **data** within structures

All DML operations are transactional in PostgreSQL:

```
BEGIN;
  INSERT INTO orders (...) VALUES (...);
  UPDATE inventory SET qty = qty - 1 WHERE ...;
COMMIT; -- Both changes committed atomically
```

INSERT — Adding Data

Basic INSERT

```
-- Insert with explicit column list (recommended)
INSERT INTO customers (first_name, last_name, email)
VALUES ('Alice', 'Johnson', 'alice@example.com');

-- Insert without column list (order must match table definition exactly --
fragile!)
INSERT INTO customers VALUES (DEFAULT, 'Bob', 'Smith', 'bob@example.com', NOW());
```

Multi-Row INSERT (Batch Insert)

```
-- Much faster than individual INSERTs
INSERT INTO products (sku, name, price, category_id) VALUES
('SKU-001', 'Widget A', 9.99, 1),
('SKU-002', 'Widget B', 14.99, 1),
('SKU-003', 'Gadget X', 49.99, 2),
('SKU-004', 'Gadget Y', 89.99, 2),
('SKU-005', 'Tool Z', 29.99, 3);
```

INSERT ... SELECT (Insert from Query)

```
-- Copy rows from one table to another
INSERT INTO customers_archive (customer_id, email, created_at)
SELECT customer_id, email, created_at
FROM customers
WHERE created_at < NOW() - INTERVAL '3 years';
```

ON CONFLICT — Upsert

ON CONFLICT handles the case where an insert would violate a unique constraint:

```
-- ON CONFLICT DO NOTHING (ignore duplicates)
INSERT INTO processed_events (event_id, payload)
VALUES ('evt-123', '{"type": "click"}')
ON CONFLICT (event_id) DO NOTHING;

-- ON CONFLICT DO UPDATE (update existing row)
INSERT INTO user_settings (user_id, theme, language)
VALUES (42, 'dark', 'en')
ON CONFLICT (user_id) DO UPDATE
    SET theme      = EXCLUDED.theme,
        language = EXCLUDED.language,
        updated_at = NOW();
-- EXCLUDED refers to the row that was attempted but conflicted

-- ON CONFLICT on a composite key
INSERT INTO inventory (warehouse_id, product_id, qty)
VALUES (1, 101, 50)
ON CONFLICT (warehouse_id, product_id) DO UPDATE
    SET qty = inventory.qty + EXCLUDED.qty;
-- Adds qty if exists, creates new record if not
```

INSERT with RETURNING

```
-- Get the generated ID back immediately
INSERT INTO orders (customer_id, total)
VALUES (42, 129.99)
RETURNING order_id, created_at;

-- Insert multiple rows and return all generated IDs
INSERT INTO tags (name)
VALUES ('postgresql'), ('database'), ('sql')
RETURNING tag_id, name;
```

UPDATE — Modifying Data

Basic UPDATE

```
-- Always use WHERE clause in UPDATE (update specific rows)
UPDATE customers
SET email = 'newemail@example.com'
WHERE customer_id = 1;

-- Update multiple columns
UPDATE products
SET price      = price * 1.10,    -- 10% price increase
```

```
    updated_at = NOW()  
WHERE category_id = 3;
```

UPDATE with Subquery

```
-- Update based on a subquery  
UPDATE customers  
SET tier = 'gold'  
WHERE customer_id IN (  
    SELECT customer_id  
    FROM orders  
    GROUP BY customer_id  
    HAVING SUM(total) > 10000  
);
```

UPDATE with JOIN (Using FROM clause)

```
-- PostgreSQL UPDATE with JOIN  
UPDATE employees e  
SET salary = salary * 1.15  
FROM departments d  
WHERE e.dept_id = d.dept_id  
    AND d.name = 'Engineering';  
  
-- Update using a CTE  
WITH top_customers AS (  
    SELECT customer_id, SUM(total) AS lifetime_value  
    FROM orders  
    GROUP BY customer_id  
    HAVING SUM(total) > 5000  
)  
UPDATE customers c  
SET tier = 'premium',  
    updated_at = NOW()  
FROM top_customers tc  
WHERE c.customer_id = tc.customer_id;
```

UPDATE with RETURNING

```
-- See what was changed  
UPDATE products  
SET stock_qty = stock_qty - 1  
WHERE product_id = 101 AND stock_qty > 0  
RETURNING product_id, name, stock_qty AS new_qty;
```

DELETE — Removing Rows

Basic DELETE

```
-- Delete specific rows (ALWAYS use WHERE in production)
DELETE FROM customers WHERE customer_id = 99;

-- Delete with multiple conditions
DELETE FROM sessions
WHERE user_id = 42
AND last_active < NOW() - INTERVAL '30 days';
```

DELETE with Subquery

```
-- Delete inactive products in categories with no active products
DELETE FROM products
WHERE product_id IN (
    SELECT p.product_id
    FROM products p
    JOIN categories c ON p.category_id = c.category_id
    WHERE c.is_active = FALSE
);
```

DELETE with USING (JOIN)

```
-- PostgreSQL DELETE with JOIN
DELETE FROM order_items oi
USING orders o
WHERE oi.order_id = o.order_id
AND o.status = 'cancelled'
AND o.created_at < NOW() - INTERVAL '1 year';
```

DELETE with RETURNING

```
-- Return deleted rows (useful for audit logging)
DELETE FROM sessions
WHERE expires_at < NOW()
RETURNING session_id, user_id, expires_at;

-- Move rows: delete + capture in a CTE, then insert elsewhere
WITH deleted AS (
    DELETE FROM orders
    WHERE status = 'cancelled' AND created_at < NOW() - INTERVAL '2 years'
    RETURNING *
)
INSERT INTO orders_archive SELECT * FROM deleted;
```

MERGE — Upsert Operations

`MERGE` was added in PostgreSQL 15 and follows the SQL:2003 standard.

```
-- Full MERGE syntax
MERGE INTO target_table AS target
USING source_table AS source
ON target.key = source.key

WHEN MATCHED AND target.status != 'archived' THEN
    UPDATE SET
        target.col1 = source.col1,
        target.updated_at = NOW()

WHEN MATCHED AND target.status = 'archived' THEN
    DELETE

WHEN NOT MATCHED THEN
    INSERT (key, col1, created_at)
    VALUES (source.key, source.col1, NOW());
```

MERGE Example: Sync Products

```
-- Sync product catalog from staging to production
MERGE INTO products AS prod
USING staging_products AS staged
ON prod.sku = staged.sku

WHEN MATCHED AND prod.price != staged.price THEN
    UPDATE SET
        price = staged.price,
        updated_at = NOW()

WHEN NOT MATCHED THEN
    INSERT (sku, name, price, category_id, created_at)
    VALUES (staged.sku, staged.name, staged.price, staged.category_id, NOW());
```

Pre-PostgreSQL 15: INSERT ON CONFLICT (Equivalent)

```
-- This covers the most common MERGE use case
INSERT INTO products (sku, name, price, category_id)
SELECT sku, name, price, category_id FROM staging_products
ON CONFLICT (sku) DO UPDATE
    SET name      = EXCLUDED.name,
        price     = EXCLUDED.price,
        category_id = EXCLUDED.category_id,
        updated_at = NOW();
```

RETURNING Clause

PostgreSQL's `RETURNING` clause lets you retrieve data from rows affected by INSERT/UPDATE/DELETE:

```
-- Get the auto-generated ID after insert
INSERT INTO users (username, email)
VALUES ('johndoe', 'john@example.com')
RETURNING user_id;

-- Get full updated row
UPDATE products SET price = price * 0.9
WHERE category_id = 1
RETURNING product_id, name, price AS discounted_price;

-- Use RETURNING in a CTE (write-then-read in one query)
WITH inserted AS (
    INSERT INTO orders (customer_id, total)
    VALUES (42, 299.00)
    RETURNING order_id, customer_id
)
INSERT INTO order_audit (order_id, customer_id, action, action_at)
SELECT order_id, customer_id, 'created', NOW()
FROM inserted;
```

ASCII Visual Diagrams

Diagram 1: DML Operation Flow

```
DML OPERATION LIFECYCLE
=====

INSERT:
  Application --> INSERT statement
    |
    v
  Constraint Check (PK, UNIQUE, FK, CHECK, NOT NULL)
    |      |
  PASS    FAIL --> Error returned to application
    |
    v
  Row added to table (with MVCC version)
    |
    v
  WAL written (durability)
    |
    v
  RETURNING clause (optional) --> Row data back to application

UPDATE:
```

WHERE clause --> finds matching rows (uses indexes if available)

|

v

Old row marked as dead (xmax set)

|

v

New row version created (xmin = current txid)

|

v

RETURNING (optional) --> new row data

DELETE:

WHERE clause --> finds matching rows

|

v

Row marked as dead (xmax set)

Physical removal done by VACUUM later

|

v

RETURNING (optional) --> deleted row data

Diagram 2: INSERT ON CONFLICT Logic

INSERT ... ON CONFLICT

=====

INSERT INTO t (pk, val) VALUES (1, 'new')

|

v

Does row with pk=1 exist?

YES

NO

|

|

v

v

DO NOTHING:

Insert row normally

Skip silently

DO UPDATE:

UPDATE t SET val = EXCLUDED.val

(EXCLUDED = the row that was attempted)

WHEN SHOULD YOU USE EACH?

DO NOTHING: Event deduplication, idempotent inserts

DO UPDATE: Cache updates, syncing from external source

Diagram 3: DELETE vs TRUNCATE

DELETE FROM orders WHERE ...

=====

|

v

TRUNCATE TABLE orders

=====

|

v

For each matching row:	Lock the table (briefly)
- Mark xmax (MVCC)	
- Write WAL for each row	v
- Trigger ON DELETE fires	Truncate data file(s)
v	v
Can be rolled back ✓	Can be rolled back ✓ (in PG)
RETURNING works ✓	RETURNING not available ✗
Can use WHERE ✓	Cannot use WHERE ✗
Slow for millions of rows	Instant regardless of size ✓
ON DELETE triggers fire ✓	Triggers do NOT fire ✗
Cascades via FK triggers ✓	CASCADE explicit ✓

Practical Examples

Example 1: Order Processing Transaction

```

BEGIN;
-- Insert new order
INSERT INTO orders (customer_id, status, total)
VALUES (42, 'pending', 0)
RETURNING order_id INTO v_order_id;

-- Insert order items
INSERT INTO order_items (order_id, product_id, qty, unit_price)
VALUES
    (v_order_id, 101, 2, 29.99),
    (v_order_id, 205, 1, 49.99);

-- Update order total
UPDATE orders
SET total = (
    SELECT SUM(qty * unit_price) FROM order_items WHERE order_id = v_order_id
)
WHERE order_id = v_order_id;

-- Decrement inventory
UPDATE products
SET stock_qty = stock_qty - oi.qty
FROM order_items oi
WHERE products.product_id = oi.product_id
    AND oi.order_id = v_order_id;

COMMIT;

```

Example 2: Soft Delete Pattern

```

-- Instead of hard delete, mark as deleted
ALTER TABLE users ADD COLUMN IF NOT EXISTS deleted_at TIMESTAMPTZ;

```

```

-- Soft delete
UPDATE users
SET deleted_at = NOW()
WHERE user_id = 42;

-- Restore
UPDATE users
SET deleted_at = NULL
WHERE user_id = 42;

-- All active users view
CREATE VIEW active_users AS
SELECT * FROM users WHERE deleted_at IS NULL;

-- Hard delete soft-deleted users after 30 days
DELETE FROM users
WHERE deleted_at < NOW() - INTERVAL '30 days'
RETURNING user_id, email;

```

PostgreSQL Commands

```

# Check rows affected by a DML statement
# psql shows "INSERT 0 1", "UPDATE 3", "DELETE 5" in the output

# Run a DML script
psql -U postgres -d mydb -f update_script.sql

# Run DML and capture output
psql -U postgres -d mydb -c "DELETE FROM old_sessions RETURNING session_id" >
deleted.csv

# Enable timing to measure DML performance
\timing on
INSERT INTO bulk_data SELECT generate_series(1, 1000000);
\timing off

```

SQL Examples

Example 1: INSERT DEFAULT Values

```

-- Columns with defaults can be omitted
CREATE TABLE events (
    event_id BIGSERIAL,
    event_type VARCHAR(50) NOT NULL,
    occurred TIMESTAMPTZ DEFAULT NOW(),
    processed BOOLEAN DEFAULT FALSE
);

```

```
-- Only provide required columns
INSERT INTO events (event_type) VALUES ('user.login');
-- occurred and processed get their defaults automatically
```

Example 2: Copy Table Data

```
-- Full table copy
INSERT INTO products_backup SELECT * FROM products;

-- Selective copy with transformation
INSERT INTO products_v2 (sku, name, price_cents)
SELECT sku, name, (price * 100)::INT
FROM products;
```

Example 3: UPDATE with Calculated Values

```
-- Compound assignment operators
UPDATE counters SET page_views = page_views + 1 WHERE page_id = 'home';
UPDATE accounts SET balance = balance - 500.00 WHERE account_id = 1;
UPDATE products SET price = ROUND(price * 1.1, 2) WHERE category_id = 2;
```

Example 4: Conditional UPDATE with CASE

```
UPDATE employees
SET salary = CASE
    WHEN performance_rating = 'excellent' THEN salary * 1.20
    WHEN performance_rating = 'good'      THEN salary * 1.10
    WHEN performance_rating = 'average'   THEN salary * 1.05
    ELSE salary
END,
updated_at = NOW();
```

Example 5: UPDATE with LIMIT (Batch Update)

```
-- Process in batches to avoid long-running transactions
UPDATE emails
SET status = 'processed'
WHERE status = 'pending'
    AND email_id IN (
    SELECT email_id FROM emails
    WHERE status = 'pending'
    ORDER BY created_at
    LIMIT 1000
);
```

Example 6: DELETE Cascade Manually

```
-- Delete in child-first order to avoid FK violations
DELETE FROM order_items WHERE order_id IN (
    SELECT order_id FROM orders WHERE customer_id = 99
);
DELETE FROM orders WHERE customer_id = 99;
DELETE FROM customers WHERE customer_id = 99;
```

Example 7: Bulk INSERT with generate_series

```
-- Insert test data
INSERT INTO users (username, email, created_at)
SELECT
    'user_' || gs,
    'user_' || gs || '@example.com',
    NOW() - (random() * INTERVAL '365 days')
FROM generate_series(1, 10000) gs;
```

Example 8: Upsert with RETURNING

```
-- Upsert and get the result
INSERT INTO page_view_counts (page_url, view_count)
VALUES ('/products', 1)
ON CONFLICT (page_url) DO UPDATE
    SET view_count = page_view_counts.view_count + 1
RETURNING page_url, view_count;
```

Example 9: DELETE Oldest Records (Retention Policy)

```
-- Enforce 90-day retention on logs
WITH deleted AS (
    DELETE FROM application_logs
    WHERE logged_at < NOW() - INTERVAL '90 days'
    RETURNING log_id
)
SELECT COUNT(*) AS deleted_count FROM deleted;
```

Example 10: Multi-Table UPDATE via CTE

```
-- Update order totals from order_items
WITH order_totals AS (
    SELECT order_id, SUM(qty * unit_price) AS total
    FROM order_items
    GROUP BY order_id
)
```

```

UPDATE orders o
SET total = ot.total,
    updated_at = NOW()
FROM order_totals ot
WHERE o.order_id = ot.order_id
    AND o.total != ot.total; -- Only update if total has changed

```

Example 11: INSERT and Archive Pattern

```

-- Atomic move: delete from source + insert into archive
WITH archived AS (
    DELETE FROM sessions
    WHERE expires_at < NOW()
    RETURNING *
)
INSERT INTO sessions_archive
SELECT *, NOW() AS archived_at FROM archived;

```

Example 12: MERGE for ETL

```

-- Daily ETL sync from staging to production
MERGE INTO dim_customers AS prod
USING (SELECT DISTINCT ON (customer_id) *
    FROM staging_customers
    ORDER BY customer_id, extracted_at DESC) AS staged
ON prod.customer_id = staged.customer_id

WHEN MATCHED AND (
    prod.name != staged.name OR
    prod.email != staged.email OR
    prod.segment != staged.segment
) THEN UPDATE SET
    name = staged.name,
    email = staged.email,
    segment = staged.segment,
    updated_at = NOW()

WHEN NOT MATCHED THEN
    INSERT (customer_id, name, email, segment, created_at)
    VALUES (staged.customer_id, staged.name, staged.email, staged.segment, NOW());

```

Common Mistakes

Mistake 1: UPDATE Without WHERE

```

-- CATASTROPHIC: Updates every row in the table
UPDATE products SET price = 0;

```

```
-- ALWAYS use WHERE
UPDATE products SET price = 0 WHERE product_id = 99;
```

Mistake 2: DELETE Without WHERE

```
-- CATASTROPHIC: Deletes every row
DELETE FROM customers;

-- Use WHERE
DELETE FROM customers WHERE customer_id = 99;
```

Mistake 3: Not Using Transactions for Multi-Step DML

```
-- BAD: If step 2 fails, step 1 already committed
INSERT INTO orders (...) VALUES (...); -- Step 1
UPDATE inventory SET qty = qty - 1 WHERE ...; -- Step 2

-- GOOD: Atomic
BEGIN;
  INSERT INTO orders (...) VALUES (...);
  UPDATE inventory SET qty = qty - 1 WHERE ...;
COMMIT;
```

Mistake 4: Large UPDATE Without Batching

Updating millions of rows in one statement holds locks for a long time and generates huge WAL. Batch instead:

```
DO $$
DECLARE batch_size INT := 10000;
BEGIN
  LOOP
    UPDATE orders SET migrated = TRUE
    WHERE order_id IN (
      SELECT order_id FROM orders
      WHERE migrated IS NULL
      LIMIT batch_size
    );
    EXIT WHEN NOT FOUND;
    PERFORM pg_sleep(0.1); -- Brief pause between batches
  END LOOP;
END $$;
```

Mistake 5: Using EXCLUDED Wrong in ON CONFLICT

`EXCLUDED` refers to the row that was proposed for insertion but conflicted. It does NOT mean "the existing row." The existing row is referenced by the table name.

```
-- EXCLUDED.price = proposed new price
-- products.price = current price in the table
INSERT INTO products (id, price) VALUES (1, 99.99)
ON CONFLICT (id) DO UPDATE
    SET price = GREATEST(EXCLUDED.price, products.price); -- Keep higher price
```

Mistake 6: Forgetting RETURNING After INSERT for Generated ID

Instead of doing an INSERT and then a separate SELECT to get the ID, use RETURNING:

```
-- BAD: Two round trips
INSERT INTO orders (...) VALUES (...);
SELECT MAX(order_id) FROM orders; -- Race condition!

-- GOOD: One round trip, safe
INSERT INTO orders (...) VALUES (...) RETURNING order_id;
```

Best Practices

1. Always include **WHERE** in **UPDATE** and **DELETE** — protect against accidental full-table operations.
2. Use **transactions for multi-step DML** — ensure all related changes succeed or all fail.
3. Use **RETURNING** to get generated IDs and avoid extra SELECT queries.
4. **Batch large DML operations** — avoid holding locks on millions of rows.
5. Use **INSERT ... ON CONFLICT** for idempotent operations — safer than checking existence first.
6. Use **DELETE ... RETURNING** to capture deleted rows for audit trails.
7. **Test DML in a transaction before committing** — `BEGIN; UPDATE ...; SELECT ...; ROLLBACK; .`
8. Use **soft deletes** for data that may need to be recovered.
9. Avoid **INSERT INTO t SELECT * FROM t** without a WHERE — may cause infinite loops with triggers.
10. Set **statement_timeout** for DML that should not run for long.

Performance Considerations

- **Multi-row INSERT** is much faster than individual INSERTs (one round-trip vs. N)
- **COPY command** is the fastest way to bulk load data (10-100x faster than INSERT for large datasets)
- **DELETE on large tables** generates WAL for every row; `TRUNCATE` is faster for removing all rows
- **Partial indexes** can dramatically speed up common UPDATE/DELETE patterns (e.g., `WHERE status = 'pending'`)
- **Disable unused indexes** during bulk loads to speed up insertions, then rebuild
- **work_mem** affects sort and hash operations in complex UPDATE/DELETE queries

```
-- Fastest bulk load: COPY
COPY products (sku, name, price) FROM '/tmp/products.csv' WITH (FORMAT CSV, HEADER);

-- Second fastest: multi-row INSERT
INSERT INTO products (sku, name, price) VALUES (...), (...), (...); -- 1000 rows at once

-- Disable triggers for bulk load
ALTER TABLE products DISABLE TRIGGER ALL;
-- ... bulk load ...
ALTER TABLE products ENABLE TRIGGER ALL;
```

Interview Questions

1. What are the DML commands in SQL?
 2. What is the difference between DELETE and TRUNCATE ?
 3. What is ON CONFLICT DO UPDATE and what is EXCLUDED ?
 4. How do you perform an upsert in PostgreSQL?
 5. What does the RETURNING clause do?
 6. How do you update rows based on data from another table?
 7. How do you move rows from one table to another atomically?
 8. What is the MERGE command and when was it added to PostgreSQL?
 9. Why should you batch large UPDATE/DELETE operations?
 10. How do you safely delete all rows from a table while keeping the structure?
-

Interview Answers

Q3: ON CONFLICT and EXCLUDED

ON CONFLICT specifies what to do when an INSERT violates a unique or exclusion constraint. DO NOTHING silently ignores the conflict. DO UPDATE runs an UPDATE on the existing row. EXCLUDED is a special pseudo-table in DO UPDATE that represents the row values that were proposed for insertion (the ones that conflicted). You use EXCLUDED.column_name to access the values from the failed INSERT.

Q7: Moving Rows Atomically

Use a writable CTE with RETURNING:

```
WITH moved AS (
    DELETE FROM source_table WHERE condition RETURNING *
)
INSERT INTO destination_table SELECT * FROM moved;
```

This is atomic — either both the DELETE and INSERT happen or neither does.

Q9: Batching Large DML

Large UPDATE/DELETE operations: acquire locks on all affected rows for the entire operation duration, generate large amounts of WAL, block VACUUM from reclaiming dead tuples, increase checkpoint pressure,

and may cause out-of-memory errors. Batching (processing 1,000-10,000 rows at a time) reduces lock contention, keeps WAL manageable, and allows VACUUM to run between batches.

Hands-on Exercises

Exercise 1: Basic CRUD

Create a `books` table. INSERT 10 books, UPDATE 3 of them (different columns), DELETE 2, and verify the final state.

Exercise 2: ON CONFLICT Practice

Create a `page_stats` table with (url, views, last_viewed). Write an UPSERT that increments views when a URL is revisited, or inserts a new record.

Exercise 3: UPDATE with JOIN

Create `employees` and `departments` tables. Update salaries for employees in the 'Engineering' department using an UPDATE with FROM (join).

Exercise 4: Bulk Operations and Batching

Insert 500,000 rows using `generate_series`. Then write a batched UPDATE that processes 10,000 rows at a time.

Exercise 5: Atomic Move

Create `jobs_pending` and `jobs_completed` tables. Write a query that atomically moves 100 completed jobs from `jobs_pending` to `jobs_completed` and returns the moved job IDs.

Solutions

Solution 2

```
CREATE TABLE page_stats (  
    url          TEXT PRIMARY KEY,  
    views        INT DEFAULT 0,  
    last_viewed  TIMESTAMPTZ DEFAULT NOW()  
);  
  
-- Simulate page visits  
INSERT INTO page_stats (url, views, last_viewed) VALUES ('/home', 1, NOW())  
ON CONFLICT (url) DO UPDATE  
    SET views      = page_stats.views + 1,  
        last_viewed = NOW();  
  
-- Verify  
SELECT * FROM page_stats;
```

Solution 3

```

CREATE TABLE departments (
    dept_id SERIAL PRIMARY KEY,
    name VARCHAR(100)
);
CREATE TABLE employees (
    emp_id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    dept_id INT REFERENCES departments(dept_id),
    salary NUMERIC(10,2)
);
INSERT INTO departments (name) VALUES ('Engineering'), ('Marketing');
INSERT INTO employees (name, dept_id, salary) VALUES
    ('Alice', 1, 80000), ('Bob', 1, 90000), ('Carol', 2, 70000);

UPDATE employees e
SET salary = salary * 1.15
FROM departments d
WHERE e.dept_id = d.dept_id AND d.name = 'Engineering';

SELECT e.name, d.name AS dept, e.salary FROM employees e JOIN departments d USING
(dept_id);

```

Solution 5

```

CREATE TABLE jobs_pending (job_id SERIAL PRIMARY KEY, payload JSONB, created_at
TIMESTAMPTZ DEFAULT NOW(), status TEXT DEFAULT 'pending');
CREATE TABLE jobs_completed (job_id INT PRIMARY KEY, payload JSONB, created_at
TIMESTAMPTZ, completed_at TIMESTAMPTZ DEFAULT NOW());

-- Insert test data
INSERT INTO jobs_pending (payload, status)
SELECT '{"task": "job_' || gs || '"}'::JSONB,
    CASE WHEN gs % 3 = 0 THEN 'done' ELSE 'pending' END
FROM generate_series(1, 200) gs;

-- Atomic move
WITH moved AS (
    DELETE FROM jobs_pending
    WHERE status = 'done'
    AND job_id IN (SELECT job_id FROM jobs_pending WHERE status = 'done' LIMIT
100)
    RETURNING job_id, payload, created_at
)
INSERT INTO jobs_completed (job_id, payload, created_at)
SELECT job_id, payload, created_at FROM moved
RETURNING job_id;

```

Advanced Notes

COPY Command for Bulk Loads

```
-- Import from CSV (fastest method)
COPY products (sku, name, price)
FROM '/path/to/products.csv'
WITH (FORMAT CSV, HEADER TRUE, DELIMITER ',', NULL '');

-- Export to CSV
COPY (SELECT * FROM orders WHERE order_date > '2024-01-01')
TO '/path/to/export.csv'
WITH (FORMAT CSV, HEADER TRUE);

-- From stdin (psql client-side copy)
\COPY products (sku, name, price) FROM '/local/file.csv' CSV HEADER
```

Writeable CTEs for Complex DML

```
-- Insert, update, and log in one statement
WITH
new_order AS (
    INSERT INTO orders (customer_id, total)
    VALUES (42, 299.00)
    RETURNING order_id, customer_id, total
),
new_items AS (
    INSERT INTO order_items (order_id, product_id, qty, unit_price)
    SELECT order_id, 101, 2, 149.50 FROM new_order
    RETURNING order_id
)
INSERT INTO audit_log (entity, entity_id, action)
SELECT 'order', order_id, 'created' FROM new_items;
```

Cross-References

- **Previous:** [01 ddl commands.md](#) — Creating database structures
- **Next:** [03 select basics.md](#) — Querying data
- **Related:** [01 Fundamentals/06 acid properties.md](#) — Transactions and ACID
- **See Also:** [04 Advanced SQL/04 cte.md](#) — Writable CTEs
- **See Also:** [07 Performance/03 bulk loading.md](#) — COPY and bulk load optimization